

Running Guide

EN2H Booking Platform API

Version 1.0 | EN2H Software Engineer Intern (NestJS) Technical Assignment

Author: Shreyanshu Maske | Date: July 2026

Live API: <https://en2h-booking-api-to69.onrender.com>

Swagger UI: <https://en2h-booking-api-to69.onrender.com/api/docs>

1. Prerequisites

Requirement	Version / Notes
Node.js	20 or later
npm	Bundled with Node.js
PostgreSQL	16+ (or use the provided Docker Compose setup, recommended)
Docker & Docker Compose	Optional but recommended — removes the need to install PostgreSQL locally
Git	To clone the repository

2. Clone the Repository

```
git clone https://github.com/shreyanshu1966/en2h_assignment.git
cd en2h_assignment
```

3. Install Dependencies

```
npm install
```

4. Configure Environment Variables

Copy the example environment file and adjust values if needed:

```
cp .env.example .env
```

Variable	Description	Default
PORT	HTTP port the API listens on	3000
DB_HOST	PostgreSQL host	localhost
DB_PORT	PostgreSQL port	5432
DB_USERNAME	PostgreSQL username	postgres
DB_PASSWORD	PostgreSQL password	postgres
DB_NAME	PostgreSQL database name	en2h_booking
JWT_SECRET	Signing secret for access tokens	— required

JWT_EXPIRES_IN	Access token lifetime	15m
JWT_REFRESH_SECRET	Signing secret for refresh tokens	— required
JWT_REFRESH_EXPIRES_IN	Refresh token lifetime	7d

5. Database Setup

Option A — Docker (recommended)

```
docker compose up -d postgres
```

This starts PostgreSQL on `localhost:5432` using the credentials from `.env`.

Option B — Local PostgreSQL install

Create a database matching `DB_NAME` and point the `DB_*` variables in `.env` at it.

6. Run Database Migrations

```
npm run migration:run
```

This applies `src/migrations/*-InitSchema.ts`, which creates the `users`, `refresh_tokens`, `services`, and `bookings` tables, the `bookings_status_enum` type, and the unique index that prevents duplicate bookings.

Script	Purpose
<code>npm run migration:run</code>	Apply all pending migrations
<code>npm run migration:revert</code>	Roll back the most recent migration
<code>npm run migration:generate -- src/migrations/<Name></code>	Generate a new migration from entity changes

7. Run the Application

Locally (development, with auto-reload)

```
npm run start:dev
```

Locally (production build)

```
npm run build  
npm run start:prod
```

With Docker Compose (API + PostgreSQL together)

```
docker compose up --build
```

The `api` container waits for PostgreSQL's healthcheck, then its entrypoint script runs migrations automatically before starting the app — no manual migration step needed in this mode.

Once running, the API listens on `http://localhost:3000` with all routes under the `/api/v1` prefix.

8. Verify It's Running

```
curl http://localhost:3000/api/v1  
# {"status":"ok","service":"en2h-booking-api","timestamp":"..."}
```

Then open `http://localhost:3000/api/docs` in a browser for the interactive Swagger UI.

9. Run Tests

Command	Description
<code>npm run test</code>	Unit tests (AuthService, BookingsService) — no database required
<code>npm run test:cov</code>	Unit tests with coverage report
<code>npm run test:e2e</code>	End-to-end smoke test — requires a running, migrated database

10. Quick End-to-End Walkthrough (via curl)

```
BASE=http://localhost:3000/api/v1
```

```
# Register
```

```
curl -X POST $BASE/auth/register -H "Content-Type: application/json" \
  -d '{"email":"jane@example.com","password":"StrongP@ssw0rd","name":"Jane Doe"}'
```

```
# Login (use the accessToken from the response below in subsequent requests)
```

```
curl -X POST $BASE/auth/login -H "Content-Type: application/json" \
  -d '{"email":"jane@example.com","password":"StrongP@ssw0rd"}'
```

```
# Create a service (replace TOKEN with the accessToken)
```

```
curl -X POST $BASE/services -H "Content-Type: application/json" \
  -H "Authorization: Bearer TOKEN" \
  -d '{"title":"Haircut","description":"Basic haircut","duration":30,"price":25}'
```

```
# Create a booking (no auth required, replace SERVICE_ID)
```

```
curl -X POST $BASE/bookings -H "Content-Type: application/json" \
  -d '{"customerName":"John Smith","customerEmail":"john@example.com","customerPhone":"+15551234567","serviceId":"SERVICE_ID","bookingDate":"2026-08-01","bookingTime":"14:30"}'
```

11. Deploying to Render (optional)

The repository includes a `render.yaml` Blueprint that provisions both the web service and a managed PostgreSQL database in one step:

1. Push the repository to GitHub (already done for this submission).
2. In the Render dashboard: **New +** → **Blueprint** → select the repository.
3. Render reads `render.yaml`, creates the database and web service, and wires the `DB_*` environment variables automatically. `JWT_SECRET` and `JWT_REFRESH_SECRET` are auto-generated, not committed.
4. On deploy, the container's entrypoint script runs migrations against the managed database before starting the app.

This project's live deployment is already running at <https://en2h-booking-api-to69.onrender.com> (Swagger at `/api/docs`). It runs on Render's free tier, so it spins down after ~15 minutes of inactivity — the first request after idling can take 30-50 seconds.

12. Troubleshooting

Symptom	Likely Cause / Fix
App fails to connect to the database	Confirm PostgreSQL is running and <code>DB_*</code> values in <code>.env</code> match. If using Docker, confirm <code>docker compose ps</code> shows <code>postgres</code> as healthy.
<code>relation "users" does not exist</code> or similar	Migrations haven't been run yet — run <code>npm run migration:run</code> .
Port 3000 already in use	Another process is bound to that port; stop it or set a different <code>PORT</code> in <code>.env</code> .
401 Unauthorized on a service/booking-management route	These routes require a valid <code>Authorization: Bearer <accessToken></code> header — log in first.
409 Conflict when creating a booking	Expected — that exact service/date/time slot is already booked and not cancelled.